

KOGI PLATFORM

Community, Spaces & Social System

Extended with Messaging · Chat · Rooms · Alerts & Notifications · Events

System Design Document — v2.0

KSPC · KRMS · KMSG · KNTF · KEVT · KORG · KFED · KSGR · KMTC · KENG.COM

Kogi Team · March 2026

Module Code	Subsystem	Primary Responsibility
KSPC	Spaces	Digital community spaces, organizations, teams, events
KRMS	Rooms	Real-time chat channels scoped to spaces, projects, and entities
KMSG	Messages & Chat	Unicast DMs · Multicast group messaging · Broadcast channels
KNTF	Alerts & Notifications	System alerts, notification fan-out, priority routing, delivery tracking
KEVT	Events Management	Event lifecycle, RSVP, scheduling, event rooms, recordings
KORG	Organizations	Autonomous orgs, collectives, cooperatives, federations, governance
KFED	Feed Service	Activity feeds, posts, timelines, content ranking
KSGR	Social Graph	Follow, subscribe, watch, connect relationship graph
KMTC	Match & Discovery	Worker matching, space recommendations, talent discovery
KENG·COM	Community Engine	Analytics, personalization, sentiment, engagement scoring

Attribute	Value
Primary Language	Go (services) · Rust (core systems) · Scala (data engine) · Angular (web)
Databases	PostgreSQL · SQLite (local) · Redis (cache/realtime) · Kafka (streams)
Protocols	REST/GraphQL (external) · gRPC (internal) · WebSocket (realtime)
Status	Design Phase — v2.0 extended spec

1. System Overview

1.1 Purpose & Vision

The Kogi Community, Spaces & Social System is the connective social tissue of the Kogi platform. It connects independent workers to one another through shared spaces, purposeful rooms, direct and group messaging, broadcast channels, event management, and a rich notifications infrastructure — all built natively on top of the Kogi Portfolio System.

This v2.0 specification extends the original community design with a fully specified messaging architecture (KMSG), a tiered alerts and notifications system (KNTF), and a comprehensive events management system (KEVT), while deepening the integrations with the kogi-engine analytics and intelligence stack and the central PortfolioSystem.

Design Philosophy: Every message, notification, room, and event is anchored to a PortfolioComponent. Social interactions are not decorative — they generate signals that flow to kogi-engine and feed the recommendation, risk, and analytics engines that power the entire platform.

1.2 Position in the Platform

Direction	Module	Data Flow
Upstream → Community	kogi-office (portfolios, projects, programs)	Portfolio items shared as showcase posts; project updates published to space feeds
Upstream → Community	kogi-studio (creative assets)	Design assets published to community showcase; creative ideas shared to spaces
Upstream → Community	kogi-bank (campaigns, finance)	Fundraising campaigns appear in feeds; treasury events trigger governance notifications
Community → Downstream	kogi-exchange (matchmaking)	Social graph + engagement data powers deal-room matching and opportunity scoring
Community → Downstream	kogi-marketplace (discovery)	Talent discovery uses space membership; reviews posted to community
Bi-directional	kogi-engine	All events streamed to engine; feed ranking, match

Direction	Module	Data Flow
		scoring, recommendations consumed back
Community → Downstream	kogi-home (dashboard)	Community notifications, mentions, DM summaries pushed to home dashboard
Foundation	PortfolioSystem (Rust)	All spaces, orgs, events, rooms are PortfolioComponents; full metadata, CRDT, governance inherited

2. KSPC — Spaces

2.1 Concept

A Space is a named digital community environment — the top-level social container on the Kogi platform. Spaces can represent teams, organizations, communities of practice, cooperative guilds, investment clubs, DAOs, or any self-defined group. Each Space contains rooms, events, a member roster, a feed, and optionally an organizational governance structure. Every Space is a PortfolioComponent (ItemType::Portfolio or SubPortfolio) and inherits the full component metadata, lifecycle, CRDT, and policy systems.

2.2 Space Types

Type	Description	Example
Community	Open or invite-only community around a topic or interest	Real Estate Investors Community
Team	Collaborative working group — squads, tribes, guilds, chapters	Design Squad Alpha
Organization	Formal legal or operational entity (LLC, Corp, Trust, Fund)	Pamoja Capital LLC
Collective	Informal resource-sharing group, skill pool	Freelance Dev Collective
Cooperative	Member-owned economic cooperative	Worker-Owned Creative Coop
DAO / Autonomous Org	Decentralized autonomous org with on-chain or platform governance	Protocol Builders DAO
Federation	Network of linked Spaces, Collectives, or Orgs	Pamoja Federation
Project Space	Space tied to a specific portfolio project or program	Brand Launch 2026 Space
Private Studio	Personal creative or research environment	Jordan's Innovation Lab

2.3 Space Data Model

Field	Type / Values	Description
space.id	UUID (ComponentId)	Platform-unique identifier; maps to PortfolioSystem ComponentId
space.type	community team org collective cooperative dao federation project studio	Space category
space.name / handle	String / @handle	Display name + unique @handle for discovery and mention
space.visibility	public private protected invite-only	Access control level
space.status	ComponentStatus (Draft · Active · Paused · Archived)	Lifecycle state from PortfolioSystem
space.members[]	Vec<{user_id, role, joined_at, stake}>	Member roster; roles: owner admin mod member contributor viewer guest
space.rooms[]	Vec<ComponentId>	Linked KRMS Room component IDs
space.events[]	Vec<ComponentId>	Linked KEVT Event component IDs
space.feed	ComponentId	Space-scoped KFED activity feed ID
space.organization	Option<ComponentId>	Optional linked KORG Organization entity
space.portfolio_links[]	Vec<ComponentId>	Linked PortfolioSystem components (projects, programs, assets)
space.toolbox_ids[]	Vec<ToolBoxId>	TMS ToolBoxes attached via PortfolioSystem v2.1
space.policies[]	Vec<PolicyId>	Governance and access policies from PortfolioSystem PolicyEngine
space.analytics	ComponentAnalytics	Engagement counters: views, followers, engagement_rate, spread
space.metadata	ComponentMetadata	id · owners · created_at · updated_at · vector_clock · version · properties

3. KRMS — Rooms

3.1 Concept

Rooms are real-time communication channels. They exist within Spaces or as standalone channels attached to portfolio items, projects, deals, events, or other platform entities. A Room is the primary synchronous and asynchronous collaboration unit. Every Room is a PortfolioComponent (Container → Binder or Custom), carrying full metadata, permissions, and lifecycle management.

3.2 Room Types

Room Type	Description	Linked Entity
Direct (DM)	Private 1:1 messaging between two users (unicast)	User ↔ User
Group	Multi-user private channel (multicast, configurable limit)	User group
Broadcast	One-to-many channel; only admins/owners publish (broadcast)	Space / Org / Channel
Space Room	Channel inside a Space (general, topic, project, announcements)	Space
Project Room	Collaborative room tied to a specific portfolio project	Project (PortfolioItem)
Portfolio Room	Coordination room for a whole portfolio or program	Portfolio / Program
Deal Room	Negotiation space for marketplace deals or exchange proposals	Exchange listing
Event Room	Temporary room for an event or meeting; auto-archivable	Event (KEVT)
Governance Room	DAO/org voting and proposal discussion room	Organization (KORG)
AI Room	Chat interface with the Oba AI assistant	System / User
Support Room	User support and help-desk threads	Platform support
Community Room	Public or semi-public	Community / Space

Room Type	Description	Linked Entity
	community discussion channel	

3.3 Room Data Model

Field	Type / Values	Description
room.id	UUID	Platform-unique identifier (PortfolioSystem ComponentId)
room.type	dm group broadcast space project portfolio deal event governance ai support community	Room category — determines messaging semantics
room.name	String	Display name
room.visibility	public private protected	Access level
room.members[]	Vec<{user_id, role, muted, last_read}>	Member roster: owner moderator member readonly bot
room.pinned_messages[]	Vec<MessageId>	High-priority pinned messages
room.linked_entity	Option<ComponentId>	Linked PortfolioComponent, Event, Exchange listing, etc.
room.settings	RoomSettings	Notification prefs, archival policy, retention rules, slow-mode
room.bots[]	Vec<BotId>	AI agents or automation bots attached to the room
room.status	active archived paused read-only	Room lifecycle state
room.metadata	ComponentMetadata	Full PortfolioSystem metadata: id, owners, vector_clock, version, policy_ids

3.4 Real-Time Architecture

Rooms use WebSocket for real-time delivery, backed by Redis pub/sub for message fanout and Kafka for durable event streaming:

- Client connects to KRMS service via WebSocket on room join
- Messages published to Redis channel for instant delivery to online subscribers
- All messages written to Kafka topic for durability, analytics, and replay
- Presence (online/typing indicators) managed via Redis TTL keys
- Offline users receive messages on reconnect via PostgreSQL message log
- Oba AI agent subscribes to AI Room channels via the Sambara agent system

4. KMSG — Messages & Chat System

New in v2.0: KMSG is the core messaging layer that defines the full unicast / multicast / broadcast delivery model, the message data model, threading, delivery tracking, and the chat protocol. It sits beneath KRMS (which defines room types) and powers all message delivery.

4.1 Messaging Delivery Model

KMSG implements three delivery semantics, each mapped to room types and use cases:

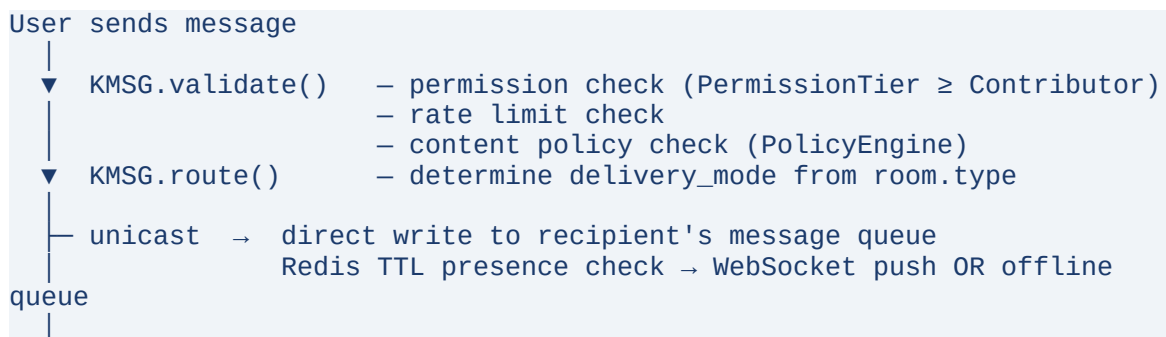
Mode	Code	Description	Room Types
Unicast	1:1	One sender → one recipient. Private, end-to-end directed. No fan-out to other parties.	DM Room
Multicast	1:N (selective)	One sender → defined set of N recipients. All recipients are known members of the channel. Fan-out to member set only.	Group Room · Space Room · Project Room · Deal Room · Event Room · Governance Room
Broadcast	1:ALL	One authorized sender → all subscribers of a channel. Subscribers may not post (read-only). Used for announcements, alerts, org-wide messages.	Broadcast Room · Space Announcements · Org Bulletin · Alert Channels

4.2 Message Data Model

Field	Type / Values	Description
message.id	UUID	Globally unique message identifier
message.room_id	ComponentId	Parent Room reference (KRMS)
message.author_id	UserId	Sending user or bot/agent ID

Field	Type / Values	Description
message.delivery_mode	unicast multicast broadcast	Delivery semantics — determined by room type
message.type	text file media link system reaction poll call embed forward	Content type
message.content	String (markdown-compatible)	Rendered content body; supports inline portfolio component embeds
message.attachments[]	Vec<Attachment>	Files, images, documents, PortfolioComponent cards
message.reactions[]	Vec<{emoji, user_ids[]}>	Emoji reactions with user ID lists
message.thread_id	Option<MessageId>	Parent message ID for threaded replies
message.mentions[]	Vec<ComponentId UserId>	Mentioned users, spaces, or portfolio components
message.reply_to	Option<MessageId>	Quoted reply reference
message.forward_from	Option<{room_id, message_id}>	Source of forwarded messages
message.status	sent delivered read failed deleted	Delivery lifecycle state
message.read_by[]	Vec<{user_id, read_at}>	Per-recipient read receipts (unicast/multicast only)
message.edited_at	Option<DateTime<Utc>>	Timestamp of last edit; edit history retained
message.metadata	ComponentMetadata (partial)	created_at · vector_clock · policy_ids

4.3 Message Delivery Pipeline



```
├─ multicast → Redis pub/sub fanout to all room members
│               WebSocket push to online members
│               Offline queue write for offline members
├─ broadcast → Redis pub/sub fanout to all channel subscribers
│               No reply capability for non-admin subscribers
└─
  ▼ Kafka write — durable event log for all delivery modes
  ▼ KENG ingest — message event → TelemetryEngine.ingestEnvelope()
  ▼ KNTF trigger — mention/keyword detection → notification dispatch
```

4.4 Direct Messaging (DM) — Unicast

DM conversations are persisted as DM Rooms (`room.type = dm`) with exactly two members. Key properties:

- End-to-end encryption option (AES-256 per conversation key, managed by `kogi-host`)
- Message requests: first DM from a non-connection goes to a 'Message Requests' queue, not the main inbox
- Read receipts: per-message `read_by[]` array with timestamps
- Typing indicators: Redis TTL key — `user:{id}:typing:{room_id}` — published to room WebSocket
- DM search: messages indexed in `SearchEngine` per user namespace; not cross-user accessible
- DM archiving: user can archive a DM thread; not deleted for other party

4.5 Group Messaging — Multicast

Group rooms support up to the configured member limit (default: 250 members). Properties:

- Member management: owner and moderators can add/remove members, adjust roles
- Message threading: any message can be replied-to, creating a thread with `thread_id`
- @mentions: mentions generate KNTF notifications for tagged users
- Pinned messages: moderators can pin up to 50 messages per room
- Group DMs: informal group conversations without a formal Space affiliation
- Rich media: images, files, portfolio component cards, link previews supported

4.6 Broadcast Messaging — Broadcast Channels

Broadcast rooms are one-to-many publication channels. Only users with `PermissionTier ≥ Manager` can publish. Subscribers receive but cannot reply (unless explicitly enabled). Use cases:

- Org-wide announcements: platform-wide or org-wide important messages
- Space bulletin boards: admin-only announcement channels within a Space
- Alert channels: automated system alerts broadcast to subscribed users (see KNTF §6)
- Campaign broadcasts: fundraising and campaign messages to opted-in supporters

- Release channels: software/product release notifications to subscribers

4.7 Message Search & History

Feature	Description
Full-text search	SearchEngine indexes message content per room; personalized re-ranking via PersonalizationEngine
Date-range filter	messages filtered by created_at range; served from PostgreSQL message log
Media / file search	Filter messages by attachment MIME type or filename
@mention search	Find all messages where a specific user or component was mentioned
Thread unroll	Retrieve full thread from a thread_id anchor message
Jump to date	Paginated scroll from a target timestamp; WebSocket cursor-based pagination
Message export	Owner/admin can export room history to JSON or Markdown

4.8 Chat Service API

Endpoint	Method	Description
/kmsg/messages	POST	Send a message to a room (all delivery modes)
/kmsg/messages/{id}	GET	Retrieve a specific message
/kmsg/messages/{id}	PATCH	Edit message content (author only; within edit window)
/kmsg/messages/{id}	DELETE	Soft-delete message (tombstone; not removed from Kafka)
/kmsg/messages/{id}/react	POST	Add/remove emoji reaction
/kmsg/messages/{id}/thread	GET	Get all threaded replies to a message
/kmsg/rooms/{id}/messages	GET	Paginated message history for a room

Endpoint	Method	Description
/kmsg/rooms/{id}/search	GET	Full-text search within a room
/kmsg/dms	POST	Initiate a DM conversation (creates DM Room if not exists)
/kmsg/broadcast	POST	Publish a broadcast message (Manager+ only)
WS /kmsg/rooms/{id}/stream	WebSocket	Real-time message stream for a room

5. KNTF — Alerts & Notifications System

New in v2.0: KNTF is the platform-wide alerts and notifications infrastructure. It handles system alerts, event-triggered notifications, mention detection, priority routing, multi-channel delivery, and user preference management. It is the single delivery broker for all notification traffic across the Kogi platform.

5.1 Notification vs Alert Distinction

Type	Purpose	Priority	Examples
Notification	Informational update; user can act or dismiss	Normal / Low	New follower, post liked, comment on post, new space member
Alert	Action-required or time-sensitive signal; must not be missed	High / Critical	Governance vote expiring, budget overrun, security event, system outage, DM request
System Alert	Platform-level event from kogi-engine or infrastructure	Critical	PortfolioHealth score critical, component state Failing, deadline miss predicted
Broadcast Alert	Admin/org-issued broadcast to a subscriber group	High	Space announcement, org bulletin, campaign launched

5.2 Notification Types

Category	Notification Type	Trigger Source
Social	New follower · New subscriber · Mention in post · Comment on post · Post liked/reacted · Post shared	KSGR · KFED
Messaging	New DM · Message request · @mention in room · Room invite · Broadcast message received	KMSG · KRMS
Space & Community	Space invite · Space join	KSPC · KFED

Category	Notification Type	Trigger Source
	approved · New space member · Post in followed space · Trending in space	
Events	Event invite · Event reminder (24h / 1h / 15min) · Event started · RSVP confirmed · Event recording ready	KEVT
Governance	New proposal · Vote opens · Quorum threshold reached · Vote closing (reminder) · Resolution passed/rejected	KORG
Portfolio	Portfolio item status change · Budget alert · Dependency blocked · Milestone reached · Risk score elevated	PortfolioSystem
System / AI	kogi-engine alert · PortfolioHealth score drop · AI recommendation ready · Oba message · System maintenance	KENG · PortfolioSystem
Economic	Payment received · Campaign contribution · Investment confirmed · Invoice due · Deal room activity	kogi-bank · kogi-exchange

5.3 Notification Data Model

Field	Type / Values	Description
notification.id	UUID	Platform-unique notification ID
notification.type	NotificationType enum	Category and sub-type (see §5.2)
notification.priority	critical high normal low	Routing priority — determines delivery channel and SLA
notification.recipient_id	UserId	Target user
notification.actor_id	Option<UserId SystemId>	User or system that triggered the notification
notification.target	{entity_type, entity_id,	The object the notification is

Field	Type / Values	Description
	entity_name}	about
notification.title / body	String	Short title + optional longer body text
notification.action_url	Option<String>	Deep-link to relevant UI context
notification.channels[]	Vec<DeliveryChannel>	Delivery channels selected by routing engine (see §5.4)
notification.status	pending delivered read dismissed failed	Delivery state
notification.delivered_at	Option<DateTime<Utc>>	Timestamp of first successful delivery
notification.read_at	Option<DateTime<Utc>>	Timestamp of user read/acknowledgement
notification.expires_at	Option<DateTime<Utc>>	TTL for time-sensitive alerts; auto-dismissed after expiry
notification.grouped_with[]	Vec<NotificationId>	Grouped notification IDs (e.g. '5 new followers' collapses 5 notifications)

5.4 Delivery Channels & Routing

Channel	Mechanism	Priority Routing	Notes
In-app (push)	WebSocket · Redis pub/sub	All priorities	Primary channel; instant delivery if user is online
Notification Feed	KFED notification feed	Normal / Low	Persistent inbox; paginated; grouped by type
Push Notification	FCM / APNs (mobile)	High / Critical	Mobile device push; requires device token registration
Email	SMTP relay	Critical (digest for Normal)	Immediate for critical; daily digest for normal priority
SMS	Twilio API	Critical only	Opt-in only; emergency alerts and governance deadlines

Channel	Mechanism	Priority Routing	Notes
Broadcast Room	KMSG broadcast	High / Critical (admin)	Platform broadcast to subscribed alert channels
Third-party bridge	Slack / Discord / WhatsApp (provider)	Per user preference	Routed via kogi-providers ProviderSystem

5.5 Notification Routing Engine

The KNTF routing engine applies per-user preferences, priority rules, and deduplication before dispatching notifications:

```

Event emitted (source system)
|
▼ KNTF.classify()    - assign NotificationType + priority
▼ KNTF.resolve()    - look up recipient_id from event payload
▼ KNTF.deduplicate() - group within 30s window if same type+actor
▼ KNTF.throttle()   - check per-user rate limits (max 50/hour normal)
▼ KNTF.preference() - load user NotificationPreference record
|                   -> check muted_senders[], muted_spaces[], quiet_hours
|                   -> check per-type channel preferences
▼ KNTF.route()       - select delivery_channels[]
▼ KNTF.dispatch()    - write to each channel adapter
|                   -> in-app: Redis pub/sub -> WebSocket
|                   -> push: FCM/APNs adapter
|                   -> email: SMTP adapter (batched for digest)
|                   -> SMS: Twilio adapter (critical only)
▼ KNTF.track()       - write notification record to PostgreSQL
▼ KENG.ingest()      - notification event -> TelemetryEngine

```

5.6 User Notification Preferences

Preference	Description
per_type_channels	Per NotificationType: which channels to use (e.g. governance -> email+inapp; social -> inapp only)
quiet_hours	Time window (start_time, end_time, timezone) during which only critical alerts are delivered
muted_senders[]	UserId list — notifications from these users are suppressed
muted_spaces[]	ComponentId list — notifications from these spaces are suppressed

Preference	Description
digest_frequency	immediate hourly daily weekly — for normal-priority notifications
push_enabled	Boolean — mobile push notification opt-in
sms_enabled	Boolean — SMS opt-in (critical only)
third_party_channels[]	Vec<ProviderId> — external channels (Slack, Discord, WhatsApp) to relay to
focus_mode	off dnd focus sleep — platform-wide mode override

5.7 Alert Escalation Model

Critical alerts that are not acknowledged within a configurable window are escalated:

- T+0: In-app push + notification feed entry
- T+5min (unread): Mobile push notification
- T+15min (unacknowledged): Email dispatch
- T+30min (unacknowledged, if SMS enabled): SMS dispatch
- T+1hr (unacknowledged): Escalate to secondary contacts if configured
- Governance vote expiry: Final reminder sent 1 hour before deadline regardless of preference settings

6. KEVT — Events Management System

New in v2.0: KEVT is the full events management system — covering event lifecycle, scheduling, RSVP management, event rooms, recordings, and post-event follow-up. Every Event is a PortfolioComponent, inheriting the full metadata, governance, and analytics stack.

6.1 Event Types

Event Type	Description	Key Features
Meeting	Scheduled video or audio call; integrates with Zoom, Google Meet	Invitees, agenda, recording, notes room
Conference	Multi-session structured gathering; has a program/schedule	Multiple sessions, breakout rooms, speaker roles, recording
Workshop	Interactive skill-building session with materials and exercises	Materials library, exercise tracking, attendance cert
Mastermind	Peer-to-peer knowledge sharing group session	Round-robin structure, topic queue, private notes
AMA	Ask Me Anything — open Q&A hosted by user or org	Question queue, upvote, moderation, recording
Launch	Product, project, or portfolio launch event	Portfolio item link, media kit, announcement broadcast
Gathering	Informal community social event	RSVP, location/link, social feed
Tagup	Quick synchronous check-in; typically 15–30 min	Status updates, blockers, recurring support
Demo Day	Showcase of projects, products, or portfolio items	Presenter slots, viewer Q&A, voting, portfolio links
Governance Vote	Formal voting event for an organization; tied to Proposal	Linked to KORG Proposal, quorum tracking, result execution

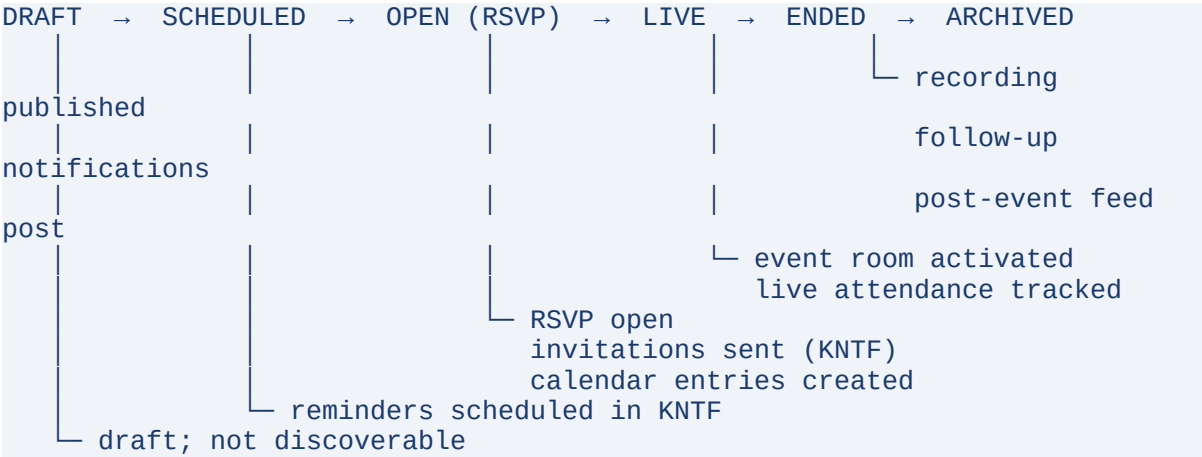
6.2 Event Data Model

Field	Type / Values	Description
event.id	UUID (ComponentId)	PortfolioSystem

Field	Type / Values	Description
		ComponentId — event is an Item
event.type	meeting conference workshop mastermind ama launch gathering tagup demo vote	Event category
event.host_id	UserId OrgId	User or Organization hosting the event
event.space_id	Option<ComponentId>	Parent Space (if event is Space-scoped)
event.title / description	String / Richtext	Display name and full event description
event.schedule	{start_time, end_time, timezone, recurrence_rule}	ISO-8601 schedule; supports rrule for recurring events
event.capacity	Option<u32>	Max participant count; None = unlimited
event.participants[]	Vec<{user_id, status, role}>	RSVP roster: status = going maybe not_going; role = host presenter attendee viewer
event.rooms[]	{main, breakout[], hallway, governance}	Linked KRMS Rooms: main event room + optional sub-rooms
event.visibility	public space invite private	Discovery and access level
event.portfolio_link	Option<ComponentId>	Linked portfolio component (project milestone, asset release, program)
event.materials[]	Vec<FileRef>	Workshop materials, slide decks, exercise files
event.recording	Option<MediaRef>	Media attachment for recorded events; published post-event
event.feed_id	ComponentId	Event-scoped activity feed for discussions and updates
event.notifications	EventNotificationConfig	Reminder schedule: 24h, 1h, 15min before; post-event follow-up

Field	Type / Values	Description
event.metadata	ComponentMetadata	Full PortfolioSystem metadata: id, owners, vector_clock, version, policy_ids

6.3 Event Lifecycle



6.4 RSVP & Attendance Management

Feature	Description
RSVP statuses	going maybe not_going waitlisted (when capacity is full)
Waitlist	Automatic waitlist management when capacity is reached; auto-promote on cancellations
Invite management	Host can invite specific users or broadcast invite to Space/Org members
Check-in	Real-time attendance check-in via event room join event; tracked as event.attendance[]
RSVP notifications	KNTF sends invite notification, 24h reminder, 1h reminder, 15min reminder
Post-event follow-up	Automated: thank you notification + recording link + feedback survey trigger
No-show tracking	Users who RSVPd 'going' but did not join tracked for future capacity planning

6.5 Event Rooms

Each Event automatically creates or links KRMS rooms based on event type:

Room	Purpose	Created When
Main Room	Primary discussion / communication channel for attendees	Event moves to SCHEDULED state
Breakout Rooms	Smaller sub-group discussion rooms; configurable count	Conference/Workshop type; created on demand by host
Hallway Room	Informal networking channel; open before/after main event	Conference/Launch/Gathering types
Governance Room	Proposal discussion + voting channel	Governance Vote event type only
Q&A Room	Moderated question queue with upvoting	AMA and Demo Day types
Host Room	Private back-channel for hosts and presenters	All multi-presenter event types

6.6 Events API

Endpoint	Method	Description
/kevt/events	POST	Create an event (DRAFT state)
/kevt/events/{id}	GET	Retrieve event details
/kevt/events/{id}	PATCH	Update event (schedule, description, capacity)
/kevt/events/{id}/publish	POST	Move to SCHEDULED; send invitations via KNTF
/kevt/events/{id}/rsvp	POST	RSVP with status: going maybe not_going
/kevt/events/{id}/checkin	POST	Record attendance on event start
/kevt/events/{id}/rooms	GET	List all rooms associated with the event
/kevt/events/{id}/recording	PUT	Upload or link event recording
/kevt/events/{id}/archive	POST	Move event to ARCHIVED; preserve recording and

Endpoint	Method	Description
		notes
/kspc/spaces/{id}/events	GET	List events in a Space

7. KFED — Feed & Social Layer

7.1 Feed Architecture

The Feed Service (KFED) powers all activity streams across the Kogi platform. Feeds are entity-scoped — personal, space, portfolio, event, org, and global — all unified through the kogi-engine PersonalizationEngine and RecommendationEngine.

7.2 Feed Types

Feed Scope	Content Sources	Personalization
Personal Home Feed	Followed users, spaces, portfolios, tags, subscriptions	Full RecommendationEngine scoring + PersonalizationEngine ranking
Space Feed	Posts within a specific Space from members	Space-scoped ranking, pinned posts, admin boosts
Portfolio Feed	Updates, milestones, status changes for a portfolio or item	Chronological + relevance; watchers + owners prioritized
Event Feed	Discussions, posts, updates tied to an Event	Chronological; event room linked
Organization Feed	Governance updates, announcements, member activity	Role-filtered visibility; admin posts pinned
Marketplace Feed	New listings, deals, offers, campaigns	Match scored by MatchEngine; personalized by trade interests
Explore / Discover Feed	Global trending content, recommended Spaces, creators	Full AI personalization; TelemetryEngine trending signals
Notification Feed	System alerts, mentions, replies, action-required items	Priority-ranked by KNTF urgency; grouped by type

7.3 Post Data Model

Field	Type / Values	Description
post.id	UUID	PortfolioSystem ComponentId
post.type	post update milestone	Content type

Field	Type / Values	Description
	announcement showcase article poll event campaign repost	
post.content	Richtext (markdown + embeds)	Body; supports portfolio component embeds, code blocks, media
post.visibility	public followers space private custom	Distribution scope
post.linked_component	Option<ComponentId>	Linked PortfolioSystem component (milestone, asset release, etc.)
post.analytics	ComponentAnalytics	Impressions, reach, engagements, CTR, view time, sentiment score
post.status	draft scheduled published archived removed	Lifecycle state

7.4 Social Interaction Primitives

Primitive	Target	KENG Signal
like / react	Post, Comment, Component	Engagement signal → AnalyticsEngine
comment	Post, Component, Event	Depth engagement → RecommendationEngine
share / repost	Post, Showcase item	Virality → TelemetryEngine
save / bookmark	Post, Component, Space	Long-term interest → PersonalizationEngine
follow	User, Space, Tag, Portfolio	Feed subscription + KSGR edge: follows
subscribe	Space, Portfolio, Creator	Paid/privileged follow; KSGR edge: subscribes
watch	Component, Project, Portfolio	Change notifications; KSGR edge: watches
tag / mention	Post, Room message	Graph edge + KNTF notification trigger
donate / invest	Component, Campaign, Org	Capital flow → kogi-bank / kogi-exchange

Primitive	Target	KENG Signal
poll / survey	Space, Post	Governance + engagement data → AnalyticsEngine

8. KORG — Organizations, Collectives & Cooperatives

8.1 Organization Types

Type	Governance Model	Description
Collective	Consensus / flat	Informal resource and skill sharing group; no formal legal entity
Cooperative	One member, one vote	Member-owned economic entity with shared profits and voting rights
Autonomous Organization	Proposal + quorum vote	Rule-governed org with codified bylaws and automated governance
DAO	Token-weighted or 1M1V	Decentralized autonomous org; optionally on-chain governance
Team	Hierarchical / lead	Operational working group within an org or space
Guild	Merit / reputation	Skill-based professional association
Federation	Federal / representative	Network of linked orgs, collectives, cooperatives
Foundation	Board / trustee	Non-profit entity; grants, donations, impact programs
Investment Club	Proportional stake voting	Group investing and portfolio management entity

8.2 Governance System

Component	Description
Proposal	Motion to take action; title, body, linked action, voting window, required quorum
Vote	Member vote: approve reject abstain; weighted by stake or 1M1V
Quorum	Minimum participation threshold; configurable per org type
Resolution	Outcome of a passed proposal; may trigger automated actions via Oba agent

Component	Description
Veto	Founder/governance-tier emergency override mechanism
Amendment	Proposal to modify org bylaws, policies, or governance rules
Treasury Proposal	Financial action requiring approval: budget allocation, investment, distribution
Membership Proposal	Add, remove, or change role of a member

9. Social Graph (KSGR)

9.1 Graph Design

The Kogi social graph is a directed, typed property graph. Nodes represent users, spaces, organizations, portfolio components, events, and posts. Edges represent typed relationships with properties such as weight, timestamp, visibility, and metadata. The kogi-engine GraphEngine (Scala) provides graph traversal, critical path analysis, dependency closure, and social analytics.

9.2 Edge Types

Edge	From → To	Properties
follows	User → User Space Tag Portfolio	since, notification_level
subscribes	User → Space Portfolio Creator	tier, since, auto_renew
watches	User → PortfolioComponent	since, alert_level → triggers KNTF watch alerts
member_of	User → Space Organization	role, joined_at, stake, status
owns	User → Component Space Org	privilege_tier, since → PortfolioSystem PermissionTier mapped
collaborates_on	User → PortfolioComponent	role, contribution_type
invested_in	User Org → Component Campaign	amount, instrument, date
linked_to	PortfolioComponent → PortfolioComponent	link_type, weight → GraphEdge in PortfolioSystem
posted_in	Post → Space Feed	visibility, pinned
part_of	Space Org → Federation	federate_since, role
messed	User → User (DM room)	room_id, last_message_at → KMSG unicast anchor

10. kogi-engine Integration

Engine Connection: Every action taken in the Community system — messages, posts, reactions, room joins, RSVP, notification delivery, governance votes — generates a StreamEvent or PlatformDataEnvelope that is ingested by kogi-engine. The engine returns scores, recommendations, and alerts back to the community layer in near-real time.

10.1 Event Ingestion — Community → kogi-engine

Community Action	Engine Ingest Method	Engine Signal
Post liked / reacted	TelemetryEngine.ingestEnvelope()	PortfolioSignal.collaboration += delta
Message sent (any mode)	AnalyticsEngine.ingest(StreamEvent)	collaboration signal; module = community
User follows / subscribes	RecommendationEngine.recordInteraction()	Social graph interaction → persona update
Space joined / left	AnalyticsEngine.ingestModuleMetric()	community module membership metric
Event RSVP / attendance	TelemetryEngine.ingestEnvelope()	Collaboration + engagement signal
Governance vote cast	AnalyticsEngine.ingest(StreamEvent)	collaboration signal; RiskEngine.score()
Notification opened	RecommendationEngine.recordInteraction()	Feedback loop: notification relevance score
Portfolio item showcased	AnalyticsEngine.ingest(StreamEvent)	productivity + cashFlow signal from portfolio
Broadcast delivered	TelemetryEngine.ingestEnvelope()	Reach + engagement telemetry

10.2 Engine Outputs → Community Layer

Engine Output	Type	Community Consumer
Feed recommendations	AnalyticsSnapshot.recommendCards[]	KFED personal feed ranking and Explore feed
Discover cards	AnalyticsSnapshot.discoverCards[]	KFED Discover feed; Space suggestions
Space match score	MatchEngine result	KMTC — worker-to-space recommendations

Engine Output	Type	Community Consumer
Collaborator match	MatchEngine result	KMTC — collaborator discovery UI
Portfolio health alert	PortfolioHealthScore (RiskEngine)	KNTF system alert → community notification
Sentiment analysis	AnalyticsEngine sentiment signal	Space health dashboard; moderation flags
Trending content	TelemetryEngine snapshot	KFED Explore trending section; hashtag leaderboard
Risk optimization plan	RiskOptimizationPlan	Oba agent community health suggestions
Search re-ranking	SearchEngine.search() personalized	KMTC search results + space search

10.3 Sub-engine Roles in Community

Sub-engine	Community Role
AnalyticsEngine	Ingests all community StreamEvents; computes engagement, reach, collaboration signals per user profile; generates community health snapshots
TelemetryEngine	Processes PlatformDataEnvelopes from community services; tracks message/post/notification flow; builds component flow statistics for the community module
RecommendationEngine	Records all social interactions as UserInteractions; builds user social profiles; powers collaborative + content-based feed ranking; cold-start handling for new users
PersonalizationEngine	Adapts feed content density, ordering, and feature flags per user segment; runs A/B experiments on notification formats and feed composition; persona lifecycle
MatchEngine	Matches users to spaces (worker-to-space); identifies collaborators (user-to-user); connects investors to campaigns; surfaces open project roles to community members
GraphEngine	Models the social graph as a directed typed graph; traverses follow/subscribe/member-of edges for social path analysis; powers

Sub-engine	Community Role
	'mutual connections' and network centrality scoring
RiskEngine	Scores community health: member activity rates, governance participation, post frequency, room message volume; flags at-risk spaces and stagnant orgs
SearchEngine	Full-text indexes posts, rooms, space names, user profiles, event titles; personalizes result ranking via persona signals; powers explore and KMTC discovery
DataStreamingEngine	In-memory event stream for community events; pub/sub subscriptions for real-time feed updates and notification dispatch

10.4 gRPC Integration Pattern

Community Go services communicate with kogi-engine via gRPC (EngineGrpcServer on port 9100):

```
// Go community service → kogi-engine ingest (message sent event)
conn, _ := grpc.Dial("kogi-engine:9100", grpc.WithInsecure())
client := enginev1.NewEngineServiceClient(conn)

resp, _ := client.Ingest(ctx, &structpb.Struct{
    Fields: map[string]*structpb.Value{
        "topic": structpb.NewStringValue("community.message.sent"),
        "source": structpb.NewStringValue("service.kmsg"),
        "target": structpb.NewStringValue("kogi-engine"),
        "payload": structpb.NewStructValue(&structpb.Struct{
            Fields: map[string]*structpb.Value{
                "room_id": structpb.NewStringValue(roomId),
                "delivery_mode": structpb.NewStringValue("multicast"),
                "author_id": structpb.NewStringValue(userId),
            },
        }),
    },
})
```

11. PortfolioSystem Integration

Foundation: All community entities — Spaces, Rooms, Events, Organizations, Posts — are PortfolioComponents at their core. The Community layer is a domain-specific projection on top of the PortfolioSystem (portfolio_system.rs v2.1). This section maps community entities to their PortfolioSystem representations and integration points.

11.1 Community Entity → PortfolioSystem Mapping

Community Entity	PortfolioSystem Representation	Key Integration Points
Space	Item { item_type: ItemType::Portfolio(...) } or SubPortfolio	ComponentMetadata, lifecycle, VectorClock, PolicyEngine, CRDT federation
Room	Container { container_type: ContainerType::Custom("Room") }	ComponentData.users (PermissionTier maps to room roles), toolbox_ids (TMS bots), action_history
Event	Item { item_type: ItemType::Project(ProjectPayload { project_type: ProjectType::Custom("Event") }) }	ItemBook.schedule (event schedule), ItemBook.charter (event description), ComponentAnalytics
Organization	Item { item_type: ItemType::Portfolio(PortfolioItemPayload { mission, kpis }) }	Governance via PolicyEngine; ResourceAllocation for treasury; ApprovalWorkflow for proposals
Post	Item { item_type: ItemType::Artifact(ArtifactPayload) }	ComponentAnalytics (engagement), action_history (likes, shares, comments), visibility field
Message	Stored in message-service DB; references room ComponentId	message.metadata.policy_id s references PortfolioSystem policies; CRDT timestamp sync
Notification	Stored in KNTF service DB; references source ComponentId	notification.target.entity_id references any PortfolioSystem ComponentId
ToolBox (TMS)	Rooms and Spaces carry	Bots and automation tools

Community Entity	PortfolioSystem Representation	Key Integration Points
	toolbox_ids (v2.1)	attached to rooms via PortfolioSystem.attach_toolbox()

11.2 Governance Bridge — KORG ↔ PortfolioSystem

Organization governance in KORG maps directly to PortfolioSystem governance primitives:

KORG Concept	PortfolioSystem Primitive	Notes
Governance Proposal	ApprovalRequest (request_approval())	proposal.id = request.id; proposal body stored in request.reason + ItemBook.charter
Vote outcome	resolve_approval(approved, resolver, notes)	Passed = ApprovalStatus::Approved → triggers automated resolution
Treasury action	allocate_budget() + record_spend()	Budget proposals allocate via ResourceAllocation; execution records consumption
Member role change	ComponentUsers.add_user(user, PermissionTier)	Role changes enacted via PortfolioSystem user management
Governance policy	PolicyEngine.register_policy_engine()	Org governance rules encoded as PolicyEngine implementations
Org portfolio	Component::Item(Item{ item_type: ItemType::Portfolio })	Every org has a linked portfolio ComponentId for asset management

11.3 Event Sourcing & Audit Trail

All community mutations generate PortfolioSystem events via the EventLog:

- Space created → PortfolioEventKind::ComponentCreated emitted; plugins notified
- Member joined → ActionKind::Join → apply_action() → ActivityLog appended
- Governance vote resolved → resolve_approval() → ApprovalGranted / ApprovalRejected event
- Budget allocated → ResourceAllocated event; consumed → ResourceConsumed event
- Content visibility changed → ActionKind::Post { visibility } → status transitions

- Version bumped → bump_version() → VersionHistory entry + ComponentMetadata.version updated

11.4 CRDT & Federation

Community spaces and organizations can participate in PortfolioFederation for multi-node deployments:

- Each kogi-host node runs an independent PortfolioSystem instance
- Space and org components replicate via CrdtLog (LWW + OR-Set) across federation peers
- Federation.sync_crdt(source, target) propagates membership and metadata changes
- Message content is NOT stored in PortfolioSystem CRDT — it lives in the message-service Kafka/PostgreSQL store
- VectorClock per component ensures causal ordering of concurrent space/member/governance changes

12. AI Workflows — Community Layer

12.1 Oba AI Assistant

The Oba AI assistant (powered by the Sambara agent system) provides intelligent community management, content creation, messaging triage, and collaboration facilitation across all community surfaces.

12.2 Content & Messaging AI Workflows

Capability	Description
Post drafting	Generates compelling post from portfolio milestone or update with tone, tags, and CTA
Showcase optimization	Reviews portfolio item descriptions and tags for discoverability improvements
DM triage	Categorizes incoming DMs as collaboration, client inquiry, or general; drafts reply suggestions
Broadcast drafting	Composes broadcast announcements from org/space context and prior engagement data
Hashtag intelligence	Analyzes trending tags and recommends which to include for maximum reach
Smart notification summarization	Summarizes notification backlog into a morning briefing digest
Event description generation	Generates event title, description, agenda, and reminder copy from host intent
Cross-platform scheduling	Plans post timing across Kogi + linked social providers based on PersonalizationEngine

12.3 Governance AI Workflows

Capability	Description
Proposal drafting	Converts natural-language intent into structured governance proposals with linked actions
Quorum monitoring	Tracks vote participation; dispatches KNTF reminders at configurable thresholds

Capability	Description
Argument summary	Summarizes governance room discussion for absent members
Resolution execution	Auto-executes passed resolutions: budget allocations, role changes, policy updates via PortfolioSystem
Governance health report	Weekly: participation rates, vote turnout, resolution velocity, trend analysis

12.4 Community Growth Triggers

Trigger	AI Action
New portfolio item published	Suggest relevant spaces to share in; draft announcement post
Engagement spike detected	Alert user; surface collaboration requests; suggest follow-up content
New space member joins	Welcome KMSG message; suggest rooms; surface relevant showcases
Event ended	Draft post-event summary; link recording; generate follow-up survey
Governance quorum missed	Re-notify via KNTF; extend window; suggest simplified proposal if appropriate
PortfolioHealth score drops (RiskEngine)	KNTF system alert + Oba community health suggestion

13. Service Architecture

13.1 Backend Services

Service	Language	Responsibilities
community-service	Go	Space CRUD, member management, governance actions
room-service	Go	Room CRUD, member management, room settings, bot attachment
message-service	Go	Message CRUD, delivery routing (unicast/multicast/broadcast), threading, attachment management, search
notification-service	Go	Notification classification, routing, dispatch (KNTF), preference management, DLQ, escalation
event-service	Go	Event lifecycle, RSVP management, event rooms, recording upload, attendance tracking
feed-service	Go	Feed assembly, fanout-on-write, feed ranking, post CRUD, notification feed
social-graph-service	Go	Follow/subscribe/watch relationship management, graph queries
post-service	Go	Post CRUD, reaction handling, comment management, content moderation hooks
community-matching-engine	Scala	Collaborator matching, space recommendations, talent discovery
community-analytics	Scala	Engagement analytics, sentiment analysis, community health scoring

13.2 Data Storage

Store	Usage
PostgreSQL	Durable storage: spaces, orgs, members, posts, messages, events, notifications, social graph edges
Redis	Real-time: presence, pub/sub fanout, session cache, feed hot data, room membership, notification delivery state, typing indicators
Kafka	Event streaming: all community events, message events, notification events, analytics pipeline input, CRDT operation log
SQLite	Local/desktop client cache: offline message queue, local feed snapshot, notification inbox cache
S3 / MinIO	Media storage: post attachments, profile images, event media, event recordings

13.3 API Endpoints

Service	Key Endpoints
Spaces (KSPC)	GET POST /kspc/spaces · /kspc/spaces/{id} · /kspc/spaces/{id}/join · /kspc/spaces/{id}/rooms · /kspc/spaces/{id}/feed · /kspc/spaces/{id}/events
Rooms (KRMS)	GET POST /krms/rooms · /krms/rooms/{id} · /krms/rooms/{id}/members · WS /krms/rooms/{id}/stream
Messages (KMSG)	POST /kmsg/messages · GET PATCH DELETE /kmsg/messages/{id} · POST /kmsg/dms · POST /kmsg/broadcast · WS /kmsg/rooms/{id}/stream
Notifications (KNTF)	GET /kntf/notifications · POST /kntf/notifications/mark-read · PUT /kntf/preferences · GET /kntf/alerts
Events (KEVT)	GET POST /kevt/events · /kevt/events/{id} · /kevt/events/{id}/publish · /kevt/events/{id}/rsvp · /kevt/events/{id}/checkin
Feed (KFED)	GET /kfed/feed · /kfed/feed/explore ·

Service	Key Endpoints
	/kfed/feed/notifications · POST /kfed/posts · GET /kfed/posts/{id}
Social Graph (KSGR)	POST DELETE /ksgr/follow · /ksgr/subscribe · GET /ksgr/followers/{id} · /ksgr/following/{id}
Organizations (KORG)	GET POST /korg/orgs · /korg/orgs/{id} · /korg/orgs/{id}/proposals · /korg/orgs/{id}/vote
Search	GET /ksrc/spaces · /ksrc/posts · /ksrc/users · /ksrc/orgs · /ksrc/explore · /ksrc/messages

14. Security, Privacy & Access Control

14.1 Access Control Model

Role	Scope	Privileges
Platform Admin	Global	Full moderation, content removal, ban, platform configuration
Space Owner	Space	Manage space config, members, rooms, events, policies; inherits PortfolioSystem Owner tier
Space Admin	Space	Manage members and rooms, moderate content; PortfolioSystem Manager tier
Space Moderator	Space	Moderate content, manage reports, mute members; PortfolioSystem Editor tier
Space Member	Space	Post, comment, react, join rooms, create events; PortfolioSystem Contributor tier
Space Viewer	Space	Read-only; no posting rights; PortfolioSystem Viewer tier
Org Governor	Organization	Propose and vote on governance; manage treasury; PortfolioSystem Manager tier
Org Member	Organization	Vote, comment, contribute; PortfolioSystem Contributor tier
Room Moderator	Room	Pin messages, remove members, configure room settings
Broadcast Publisher	Broadcast Room	Post messages to broadcast channel; PortfolioSystem Manager+ tier

14.2 Content Visibility Model

Visibility	Who Can See
public	Anyone, including unauthenticated visitors
followers	Followers and subscribers of the author/space
protected	Approved followers only
space	Space members only
organization	Org members with appropriate role
private	Explicitly mentioned parties; owner only
custom	Rule-based access list defined by owner via PolicyEngine

14.3 Data Privacy & Compliance

- GDPR right-to-erasure: complete post, message, and social graph data deletion on request
- Data minimization: only social interaction data required for function is collected
- Consent management: granular consent controls per data type and integration
- Encryption at rest and in transit: TLS 1.3 for all connections; AES-256 for stored message content
- DM end-to-end encryption: optional per-conversation encryption key management via kogi-host
- Network namespace isolation: community services operate in isolated network segments
- Audit logging: all moderation, governance, and administrative actions logged to PortfolioSystem EventLog and immutable audit store

15. Third-Party Integrations

15.1 Social Platform Integrations

Provider	Integration Type	Capabilities
Facebook / Instagram	OAuth2 + API	Cross-post content, import followers, sync events
YouTube	OAuth2 + API	Embed video, sync channel activity, cross-publish; event recording upload
WhatsApp	Business API	Broadcast messages, customer communication bridge, KNTF relay
Slack	OAuth2 + Webhooks	Cross-team notifications, channel mirroring, DM bridge
Zoom	OAuth2 + API	Event room video calls, meeting scheduling integration
Twitter / X	OAuth2 + API	Cross-post announcements, track mentions, import audience
LinkedIn	OAuth2 + API	Professional profile sync, job post distribution, network import
Discord	Bot API	Community server bridging, role sync, notification relay
Gmail	OAuth2 + IMAP	Email-to-room bridge, DM-to-email relay, KNTF email delivery
Twilio	API	SMS KNTF relay, 2FA, event reminders (critical alerts)

16. MVP Feature Scope

Feature	Module	Priority
Space creation and management	KSPC	P0 — Core
Space member management (join, invite, roles)	KSPC	P0 — Core
Direct messaging — unicast DMs	KMSG	P0 — Core
Group messaging — multicast rooms	KMSG	P0 — Core
In-app notifications and mention alerts	KNTF	P0 — Core
Post creation and personal feed	KFED	P0 — Core
Follow / unfollow users and spaces	KSGR	P0 — Core
Like, comment, share on posts	KFED	P0 — Core
Broadcast messaging — announcement channels	KMSG	P1 — High
Notification preferences and quiet hours	KNTF	P1 — High
Event creation, scheduling, and RSVP	KEVT	P1 — High
Event rooms (main + breakout)	KEVT + KRMS	P1 — High
Event reminders (KNTF integration)	KNTF + KEVT	P1 — High
Activity feed per space	KFED	P1 — High
Portfolio showcase posts	KFED	P1 — High
Basic organization creation (collective, team)	KORG	P1 — High
Space discovery and explore feed	KMTC	P1 — High
kogi-engine analytics integration	KENG·COM	P1 — High

Feature	Module	Priority
Notification escalation (push/email)	KNTF	P2 — Medium
Governance proposals and voting	KORG	P2 — Medium
Message search and history export	KMSG	P2 — Medium
AI post drafting and DM triage (Oba)	AI Layer	P2 — Medium
Event recordings and post-event follow-up	KEVT	P2 — Medium
DAO / cooperative governance model	KORG	P2 — Medium
Third-party social platform integrations	Providers	P3 — Later
Federation of spaces and orgs	KSPC / KORG	P3 — Later
DM end-to-end encryption	KMSG + kogi-host	P3 — Later
Advanced analytics dashboard	KENG·COM	P3 — Later

Appendix — Terminology Reference

Term	Definition
Space (KSPC)	Named digital community environment; top-level social container; PortfolioSystem Item
Room (KRMS)	Real-time/async communication channel; exists within or independent of a Space; PortfolioSystem Container
Message (KMSG)	Content unit sent within a Room; carries delivery_mode (unicast multicast broadcast)
Unicast	One-to-one directed message delivery (DM)
Multicast	One-to-N delivery to defined member set (Group Room, Space Room)
Broadcast	One-to-all delivery to subscribers; recipients cannot reply unless enabled
Notification (KNTF)	Informational update delivered via routing engine based on user preferences
Alert (KNTF)	High/critical priority signal requiring user acknowledgement; escalated if unread
Event (KEVT)	Scheduled gathering with lifecycle, RSVP, rooms, and recording; PortfolioSystem Item
Feed (KFED)	Activity stream scoped to user, space, portfolio, event, or global explore
Post	Content item published to a feed; PortfolioSystem Artifact Item
Social Graph (KSGR)	Directed typed property graph of all relationships: follows, subscribes, watches, member_of
Organization (KORG)	Governed group entity: collective, cooperative, DAO, team, guild, federation
PortfolioSystem	The central Rust domain engine (portfolio_system.rs v2.1) — all community entities are PortfolioComponents
kogi-engine	Scala analytics and intelligence engine; consumes all community events via gRPC/Kafka; returns scores and recommendations
Oba	The Kogi platform AI assistant running on

Term	Definition
	the Sambara agent system
VectorClock	Per-component distributed logical clock used for causal ordering of concurrent mutations
CRDT	Conflict-free replicated data type (LWW + OR-Set) used for federated community state convergence
TMS (ToolBoxId)	Tool Management System integration; rooms and spaces carry opaque ToolBoxId references for bots and automation